

Startup Matchmaking: Automation Approach

Task 2 write-up: approach and architecture

Today, matching an incoming request to suitable startups means opening a 600+ row Excel file and manually checking it against whatever the requester asked for. That takes about 7 hours a week across 10+ requests. The goal here is to turn that process, read the email, search the sheet, write a reply, into a short pipeline with one required human checkpoint before anything goes out. We already run and fully control a Hetzner server for the startin.lv website, so this tool is designed to run there as its own self-hosted service, rather than adding a stack of new paid subscriptions.

Architecture and data flow

1. Intake	Requests arrive at the team's existing mailbox. A small script checks it every few minutes and passes new messages to the matchmaking app, so there is no separate inbound-email service to pay for. Duplicate emails are ignored automatically.
2. Understand the ask	A fast, inexpensive model reads the free-text email and pulls out structured criteria: sector, stage, geography, ticket size, and deadline. Each value is checked against the known sector and stage lists; anything it doesn't recognise is reported rather than invented. It also separates must-have requirements from nice-to-haves. If a request is unclear, the system drafts a clarifying question instead of guessing.
3. Match	Startups that fail a must-have criterion are excluded. The rest are ranked by a rules-based scoring system, the same approach used in the Task 1 prototype submitted alongside this document, so the reasoning behind every rank stays visible instead of hidden in a model. If fewer than five startups qualify, the search widens automatically, relaxing nice-to-haves first and then stage or geography, and the shortlist is labelled as a wider search rather than silently returning weak matches. A genuinely empty result goes straight to a person.
4. Shortlist draft	The top 10 to 15 startups are shown, each with its name, one-line description, sector and stage, website link, and one sentence on why it fits, written by a stronger model once the shortlist is set. Where startups fit equally well, the one shortlisted less often recently is preferred, so the same well-known names don't win every time. The list is saved as a link the requester can reopen later.
5. Human review	A team member checks the list, edits it if needed, and approves it, either inside the app or directly from the notification email so nothing new has to be learned. Nothing is sent to a requester without this step.
6. Send and log	The approved shortlist goes out through the team's existing mailbox. A bounce reopens the request. Every request, shortlist, and edit is logged for future reference.

The guiding principle throughout is that a deterministic layer, not an AI model, always decides who is eligible and how they rank; AI is used only to read free text and write short explanations, never to make the final decision to send something out, and any text pulled from emails or websites is treated purely as information, never as an instruction to follow. It runs as its own service on the existing Hetzner server, which needs a small upgrade to 4 vCPU and 8GB so the two apps never compete for memory.

The 600-row sheet becomes a proper table with a fixed set of fields: name, website, one-line description, sector tags, stage, founded year, team size, funding raised, geography, contact, and last-confirmed date. Sector and stage move to controlled lists with synonyms mapped once, so “fintech” and “finanšu tehnoloģijas” resolve to the same tag, because free text in those two columns is what makes today's spreadsheet search miss good matches. Anything the import can't map is flagged for a person rather than guessed.

Startups confirm once that they can be shared with investors and event organisers, and that consent is stored with a date. A startup can opt out of specific request types, for example investor deal flow, and where a request is for an introduction rather than a list, the startup is asked before its contact details go out.

Tools and estimated running cost

Item	Tool	Est. monthly cost
Hosting (both apps)	Hetzner VPS, upgraded to 4 vCPU / 8GB so the website and the matchmaking app run as two independent services	About €7.70 (up from about €4.35 today)
Database, email, and matching logic	All reuse infrastructure already in place: Postgres, the existing mailbox, and rules that run inside the app	€0 new
AI (reading requests, writing rationale)	Claude: a fast model for reading and structuring each request, a stronger model for writing the shortlist rationale, roughly 50 requests a month	€10 to €25
Total new cost		roughly €13 to €28 a month

That is a small fraction of the roughly 28 hours a month this process currently costs the team, while keeping everything on infrastructure the team already owns and understands.

Verifying match quality, and keeping a human in the loop

- A person always reviews the shortlist, and can edit it, before anything is sent. The system never emails a requester on its own.

- Before launch, we would test the system against about 20 real past requests, including tricky or no-match cases. Half would tune the scoring; the other half would be kept aside purely to check accuracy, tracking what share of suggested startups are actually suitable and treating any missed must-have requirement as a bug to fix immediately, not just a quality issue.
- During the first three months, we would check a sample of shortlists every week against what a staff member would have chosen by hand, and track how long each request actually takes, aiming to bring it down from about 7 hours a week to about 2.
- Alongside time and precision, we would track whether it actually works for the requester: the share of shortlists that lead to a reply or an introduction, and the share of requests answered within 24 hours.

Keeping the database current

- Startups get a link every few months to review and update their own listing. Changes go into a review queue rather than appearing live immediately.
- Any listing that hasn't been confirmed in 6 months is flagged and ranked lower until someone updates it.
- Whenever a staff member manually adds or removes a startup from a suggested shortlist, that correction is recorded, so repeated corrections can point to a rule that needs adjusting.
- A monthly view shows which members have never been shortlisted, so representation gaps can be closed deliberately instead of the same names appearing every time.

What ships in Week 1 vs. Month 3

	Week 1	Month 3
Where it runs	A lightweight prototype added to the current server, to get something working quickly	Its own separate app, running on the upgraded server
Database	The Excel file imported into a proper table with fixed fields and controlled sector/stage lists	The same data, kept up to date automatically
Matching	A staff member types the request into a simple search screen and gets a ranked list in seconds, instead of scrolling through Excel	Incoming emails are read and matched automatically, with nothing retyped
Email handling	Still read and handled by a person	Incoming emails are checked automatically; sending a shortlist is a single click
Quality check	None yet. This stage establishes how long manual matching takes, and how often it leads to a reply, as a baseline	Precision, review time, and reply rate tracked weekly against that baseline
Outcome	Removes most of the manual searching straight away, with very little build risk	Two apps running smoothly side by side, and very little manual work left per request

The main risks are a bad data import, a spike in requests, or the one person who knows the system being unavailable. All three are handled by the same human-review step, straightforward monitoring, and documentation that doesn't depend on one person's memory.